

Pages Router > ... > Deploying > Static Exports

Static Exports

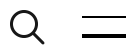
Next.js enables starting as a static site or Single-Page Application (SPA), then later optionally upgrading to use features that require a server.

When running `next build`, Next.js generates an HTML file per route. By breaking a strict SPA into individual HTML files, Next.js can avoid loading unnecessary JavaScript code on the client-side, reducing the bundle size and enabling faster page loads.

Since Next.js supports this static export, it can be deployed and hosted on any web server that can serve HTML/CSS/JS static assets.

Good to know: We recommend using the App Router for enhanced static export support.

Configuration



To enable a static export, change the output mode inside `next.config.js`:

> Menu

JS next.config.js



```
1  /**
2   * @type {import('next').NextConfig}
3   */
4  const nextConfig = {
5    output: 'export',
6  }
```

```
7    // Optional: Change links `/me` -> `/me/` and emit `/me.html` -> `/me/index.html`
8    // trailingSlash: true,
9
10   // Optional: Prevent automatic `/me` -> `/me/`, instead preserve `href`
11   // skipTrailingSlashRedirect: true,
12
13   // Optional: Change the output directory `out` -> `dist`
14   // distDir: 'dist',
15 }
16
17 module.exports = nextConfig
```

After running `next build`, Next.js will produce an `out` folder which contains the HTML/CSS/JS assets for your application.

You can utilize `getStaticProps` and `getStaticPaths` to generate an HTML file for each page in your `pages` directory (or more for [dynamic routes](#)).

Supported Features

The majority of core Next.js features needed to build a static site are supported, including:

- [Dynamic Routes when using `getStaticPaths`](#)
- Prefetching with `next/link`
- Preloading JavaScript
- [Dynamic Imports](#)
- Any styling options (e.g. CSS Modules, styled-jsx)
- [Client-side data fetching](#)
- `getStaticProps`
- `getStaticPaths`

Image Optimization

Image Optimization through `next/image` can be used with a static export by defining a custom image loader in `next.config.js`. For example, you can optimize images with a service like Cloudinary:

```
JS next.config.js

1  /** @type {import('next').NextConfig} */
2  const nextConfig = {
3    output: 'export',
4    images: {
5      loader: 'custom',
6      loaderFile: './my-loader.ts',
7    },
8  }
9
10 module.exports = nextConfig
```

This custom loader will define how to fetch images from a remote source. For example, the following loader will construct the URL for Cloudinary:

```
TS my-loader.ts TypeScript

1  export default function cloudinaryLoader({
2    src,
3    width,
4    quality,
5  }: {
6    src: string
7    width: number
8    quality?: number
9  }) {
10   const params = ['f_auto', 'c_limit', `w_${width}`, `q_${quality || 'auto'}`]
11   return `https://res.cloudinary.com/demo/image/upload/${params.join(
12     ',',
13   )}${src}`
14 }
```

You can then use `next/image` in your application, defining relative paths to the image in Cloudinary:

TS app/page.tsx

TypeScript ▾



```
1  import Image from 'next/image'
2
3  export default function Page() {
4    return <Image alt="turtles" src="/turtles.jpg" width={300} height={300} />
5  }
```

Unsupported Features

Features that require a Node.js server, or dynamic logic that cannot be computed during the build process, are **not** supported:

- [Internationalized Routing](#)
- [API Routes](#)
- [Rewrites](#)
- [Redirects](#)
- [Headers](#)
- [Middleware](#)
- [Incremental Static Regeneration](#)
- [Image Optimization](#) with the default `loader`
- [Draft Mode](#)
- `getStaticPaths` with `fallback: true`
- `getStaticPaths` with `fallback: 'blocking'`
- `getServerSideProps`

Deploying

With a static export, Next.js can be deployed and hosted on any web server that can serve HTML/CSS/JS static assets.

When running `next build`, Next.js generates the static export into the `out` folder. For example, let's say you have the following routes:

- `/`
- `/blog/[id]`

After running `next build`, Next.js will generate the following files:

- `/out/index.html`
- `/out/404.html`
- `/out/blog/post-1.html`
- `/out/blog/post-2.html`

If you are using a static host like Nginx, you can configure rewrites from incoming requests to the correct files:

nginx.conf

```
1  server {
2    listen 80;
3    server_name acme.com;
4
5    root /var/www/out;
6
7    location / {
8        try_files $uri $uri.html $uri/ =404;
9    }
10
11    # This is necessary when `trailingSlash: false`.
12    # You can omit this when `trailingSlash: true`.
13    location /blog/ {
14        rewrite ^/blog/(.*)$ /blog/$1.html break;
15    }
16
17    error_page 404 /404.html;
18    location = /404.html {
19        internal;
```

```
20   }  
21 }
```

Version History

Version	Changes
v14.0.0	<code>next export</code> has been removed in favor of <code>"output": "export"</code>
v13.4.0	App Router (Stable) adds enhanced static export support, including using React Server Components and Route Handlers.
v13.3.0	<code>next export</code> is deprecated and replaced with <code>"output": "export"</code>

Was this helpful?    

	Resources	More	About Vercel	Legal
	Docs	Next.js Commerce	Next.js + Vercel	Privacy Policy
	Learn	Contact Sales	Open Source Software	
	Showcase	GitHub	GitHub	
	Blog	Releases	X	
	Analytics	Telemetry		
	Next.js Conf	Governance		

DX Platform

Subscribe to our newsletter

Stay updated on new releases and features, guides, and case studies.

© 2024 Vercel, Inc.

